```
!pip install cltk
```

Show hidden output

```
import io

def get_max_depth(sentence):
    # This logic finds how many layers of 'recursion' are inside a node
    # It counts the height of specific linguistic markers: acl, ccomp, advcl
    # For Vedic Sanskrit, 'relcl' is the main indicator of recursion.
    # In Bengali, 'ccomp' indicates higher depth.

    heads = {}
    for line in sentence.split('\n'):
        if line.startswith('#') or not line: continue
        parts = line.split('\t')
        node_id, head_id, dep_rel = parts[0], parts[6], parts[7]
        heads[node_id] = (head_id, dep_rel)

    depths = []
    recursive_labels = ['acl:relcl', 'ccomp', 'advcl']

    for node_id, info in heads.items():
        curr_node = node_id
        d = 0
        while curr_node != '0':
            parent, label = heads[curr_node]
            if label in recursive_labels:
                d += 1
            curr_node = parent
        depths.append(d)

    return max(depths) if depths else 0

# Your task: Download the 'Sanskrit-Vedic' raw conllu file
# and input it here to generate the "Ancient Complexity Index".
```

```
# Create a folder for our data
!mkdir treebanks

# Download the Vedic Sanskrit (OIA) Treebank
!wget -O treebanks/sa_vedic.conllu https://raw.githubusercontent.com/UniversalDependencies/UD_Sanskrit-Vedic/master/sa_ved

# Download the Bengali (NIA) Treebank
!wget -O treebanks/bn_bengali.conllu https://raw.githubusercontent.com/UniversalDependencies/UD_Bengali-PUD/master/bn_pud-
```

```
--2026-05-22 16:20:19--  https://raw.githubusercontent.com/UniversalDependencies/UD_Sanskrit-Vedic/master/sa_vedic-ud-test.c
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3035277 (2.9M) [text/plain]
Saving to: ‘treebanks/sa_vedic.conllu’

treebanks/sa_vedic. 100%[===================>]   2.89M  --.-KB/s    in 0.08s

2026-05-22 16:20:19 (35.2 MB/s) - ‘treebanks/sa_vedic.conllu’ saved [3035277/3035277]

--2026-05-22 16:20:19--  https://raw.githubusercontent.com/UniversalDependencies/UD_Bengali-PUD/master/bn_pud-ud-test.conllu
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 404 Not Found
2026-05-22 16:20:19 ERROR 404: Not Found.
```

```
import os

def calculate_complexity(filename):
    with open(filename, 'r', encoding='utf-8') as f:
        data = f.read().split('\n\n')

    depths = []
    sentence_count = 0

    # These labels indicate clausal boundaries (Recursion points)
    recursive_labels = ['acl:relcl', 'ccomp', 'advcl', 'xcomp', 'acl']

    for sentence in data:
        if not sentence.strip(): continue
        sentence_count += 1
```

```
            heads = {}
            for line in sentence.split('\n'):
                if line.startswith('#') or not line: continue
                parts = line.split('\t')
                try:
                    node_id, head_id, dep_rel = parts[0], parts[6], parts[7]
                    heads[node_id] = (head_id, dep_rel)
                except IndexError:
                    continue

            # Find max recursion depth for THIS sentence
            sent_depths = []
            for node_id, info in heads.items():
                curr_node = node_id
                d = 0
                while curr_node in heads and heads[curr_node][0] != '0':
                    parent, label = heads[curr_node]
                    if any(x in label for x in recursive_labels):
                        d += 1
                    curr_node = parent
                sent_depths.append(d)

            depths.append(max(sent_depths) if sent_depths else 0)

        avg_depth = sum(depths) / len(depths) if depths else 0
        max_depth_found = max(depths) if depths else 0

        return avg_depth, max_depth_found, sentence_count

    # RUN ANALYSIS
    oia_avg, oia_max, oia_count = calculate_complexity('treebanks/sa_vedic.conllu')
    nia_avg, nia_max, nia_count = calculate_complexity('treebanks/bn_bengali.conllu')

    print(f"{'Era':<20} | {'Avg Recursion Depth':<20} | {'Deepest Cluster':<15} | {'Sentences'}")
    print("-" * 75)
    print(f"{'Old Indo-Aryan (Vedic)':<20} | {oia_avg:<20.4f} | {oia_max:<15} | {oia_count}")
    print(f"{'New Indo-Aryan (Bengali)':<20} | {nia_avg:<20.4f} | {nia_max:<15} | {nia_count}")
```

```
Era                  | Avg Recursion Depth  | Deepest Cluster | Sentences
---------------------------------------------------------------------------
Old Indo-Aryan (Vedic) | 0.6349               | 4               | 2709
New Indo-Aryan (Bengali) | 0.0000               | 0               | 0
```

```
# Clear the failed Bengali file and try these reliable links
!rm -rf treebanks
!mkdir treebanks

# Download Vedic Sanskrit (Already working)
!wget -O treebanks/sa_vedic.conllu https://raw.githubusercontent.com/UniversalDependencies/UD_Sanskrit-Vedic/master/sa_ved

# FIXED Bengali link (Direct URL from UD distribution)
!wget -O treebanks/bn_bengali.conllu https://raw.githubusercontent.com/UniversalDependencies/UD_Bengali-PUD/master/bn_pud-

# Hindi-HDTB link (Highly reliable/large)
!wget -O treebanks/hi_hindi.conllu https://raw.githubusercontent.com/UniversalDependencies/UD_Hindi-HDTB/master/hi_hdtb-ud-
```

```
--2026-05-22 16:22:04--  https://raw.githubusercontent.com/UniversalDependencies/UD_Sanskrit-Vedic/master/sa_vedic-ud-test.c
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.110.133, 185.199.108.133, 185.199.111.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.110.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3035277 (2.9M) [text/plain]
Saving to: ‘treebanks/sa_vedic.conllu’

treebanks/sa_vedic. 100%[===================>]   2.89M  --.-KB/s    in 0.08s

2026-05-22 16:22:04 (35.9 MB/s) - ‘treebanks/sa_vedic.conllu’ saved [3035277/3035277]

--2026-05-22 16:22:04--  https://raw.githubusercontent.com/UniversalDependencies/UD_Bengali-PUD/master/bn_pud-ud-test.conllu
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 404 Not Found
2026-05-22 16:22:04 ERROR 404: Not Found.

--2026-05-22 16:22:04--  https://raw.githubusercontent.com/UniversalDependencies/UD_Hindi-HDTB/master/hi_hdtb-ud-test.conllu
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 6079732 (5.8M) [text/plain]
Saving to: ‘treebanks/hi_hindi.conllu’

treebanks/hi_hindi. 100%[===================>]   5.80M  --.-KB/s    in 0.09s

2026-05-22 16:22:05 (61.2 MB/s) - ‘treebanks/hi_hindi.conllu’ saved [6079732/6079732]
```

```
import os

def calculate_complexity(filename):
    if not os.path.exists(filename) or os.path.getsize(filename) < 100:
        return 0, 0, 0 # File download error check

    with open(filename, 'r', encoding='utf-8') as f:
        content = f.read()
        data = content.split('\n\n')

    depths = []
    sentence_count = 0
    recursive_labels = ['acl:relcl', 'ccomp', 'advcl', 'xcomp', 'acl', 'conj'] # Added 'conj' for coordinate recursion

    for sentence in data:
        if not sentence.strip() or sentence.startswith('<!DOCTYPE html>'): continue
        sentence_count += 1
        heads = {}
        for line in sentence.split('\n'):
            if line.startswith('#') or not line: continue
            parts = line.split('\t')
            if len(parts) < 10: continue
            node_id, head_id, dep_rel = parts[0], parts[6], parts[7]
            heads[node_id] = (head_id, dep_rel)

        sent_depths = []
        for node_id in heads:
            curr_node = node_id
            d = 0
            visited = set()
            while curr_node in heads and heads[curr_node][0] != '0' and curr_node not in visited:
                visited.add(curr_node)
                parent, label = heads[curr_node]
                if any(x in label for x in recursive_labels):
                    d += 1
                curr_node = parent
            sent_depths.append(d)

        depths.append(max(sent_depths) if sent_depths else 0)

    return (sum(depths) / len(depths)), max(depths), len(depths)

# EXECUTE
print(f"{'Era':<25} | {'Avg Depth':<12} | {'Sentences'}")
print("-" * 55)

for label, path in [("Vedic (OIA)", "treebanks/sa_vedic.conllu"),
                    ("Bengali (NIA)", "treebanks/bn_bengali.conllu"),
                    ("Hindi (NIA)", "treebanks/hi_hindi.conllu")]:
    avg_d, max_d, count = calculate_complexity(path)
    if count > 0:
        print(f"{label:<25} | {avg_d:<12.4f} | {count}")
    else:
        print(f"{label:<25} | {'ERROR':<12} | {count}")
```

```
Era                       | Avg Depth    | Sentences
-------------------------------------------------------
Vedic (OIA)               | 0.8590       | 2709
Bengali (NIA)             | ERROR        | 0
Hindi (NIA)               | 0.7464       | 1684
```

```
!pip install transformers torch

from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

# We will use a smaller model (GPT-2 or Llama) to demonstrate the logic locally in Colab
# If you have an OpenAI API key, we can use that for massive models later
model_id = "gpt2" # Placeholder for probing logic
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(model_id)

def calculate_perplexity(sentence):
    inputs = tokenizer(sentence, return_tensors="pt")
    with torch.no_grad():
        outputs = model(**inputs, labels=inputs["input_ids"])
    loss = outputs.loss
    return torch.exp(loss).item()

# TEST SENTENCES (Identified from your Deepest Clusters)
# 1. Low Recursion (Hindi)
```

```
sent_1 = "मैंने कहा कि वह आएगा।"
# 2. High Recursivity (Hindi)
sent_2 = "मैंने कहा कि उसने सुना कि वह समझ गया कि मामला खत्म हो गया।"

print(f"Perplexity (Modern NIA Simple): {calculate_perplexity(sent_1):.4f}")
print(f"Perplexity (Modern NIA Recursive): {calculate_perplexity(sent_2):.4f}")
```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (5.0.0)
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.10.0+cpu)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers) (3.29.0)
Requirement already satisfied: huggingface-hub<2.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (1
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (1.26.4)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (26.1)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2025.11.3)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0
Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dist-packages (from transformers) (0.24.0)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.7.0)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers) (4.67.3)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0->transform
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->h
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub<2.0,>=1.
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub<2.
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub<2.0,>=1.
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub<
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer->h
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer-
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=12.3
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
config.json: 100%                                                  665/665 [00:00<00:00, 53.0kB/s]

tokenizer_config.json: 100%                                                  26.0/26.0 [00:00<00:00, 2.02kB/s]

vocab.json:        1.04M/? [00:00<00:00, 16.4MB/s]

merges.txt:        456k/? [00:00<00:00, 12.7MB/s]

tokenizer.json:        1.36M/? [00:00<00:00, 19.1MB/s]

model.safetensors: 100%                                                  548M/548M [00:04<00:00, 276MB/s]

Loading weights: 100%                                                  148/148 [00:00<00:00, 415.06it/s, Materializing param=transformer.wte.weight]
GPT2LMHeadModel LOAD REPORT from: gpt2
Key                  | Status     |  |
```

```
!rm -rf treebanks
!mkdir treebanks

# 1. OIA: Sanskrit Vedic
!wget -O treebanks/sa_vedic.conllu https://raw.githubusercontent.com/UniversalDependencies/UD_Sanskrit-Vedic/master/sa_vedi

# 2. NIA: Hindi (High quality anchor)
!wget -O treebanks/hi_hindi.conllu https://raw.githubusercontent.com/UniversalDependencies/UD_Hindi-HDTB/master/hi_hdtb-ud-

# 3. NIA: Bengali (Using BRU variant for consistency)
!wget -O treebanks/bn_bru.conllu https://raw.githubusercontent.com/UniversalDependencies/UD_Bengali-BRU/master/bn_bru-ud-te

# 4. NIA: Marathi (Comparative support)
!wget -O treebanks/mr_marathi.conllu https://raw.githubusercontent.com/UniversalDependencies/UD_Marathi-UFAL/master/mr_ufal
```

```
--2026-05-22 16:36:39--  https://raw.githubusercontent.com/UniversalDependencies/UD_Sanskrit-Vedic/master/sa_vedic-ud-test.c
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3035277 (2.9M) [text/plain]
Saving to: ‘treebanks/sa_vedic.conllu’

treebanks/sa_vedic. 100%[===================>]   2.89M  --.-KB/s    in 0.08s
```

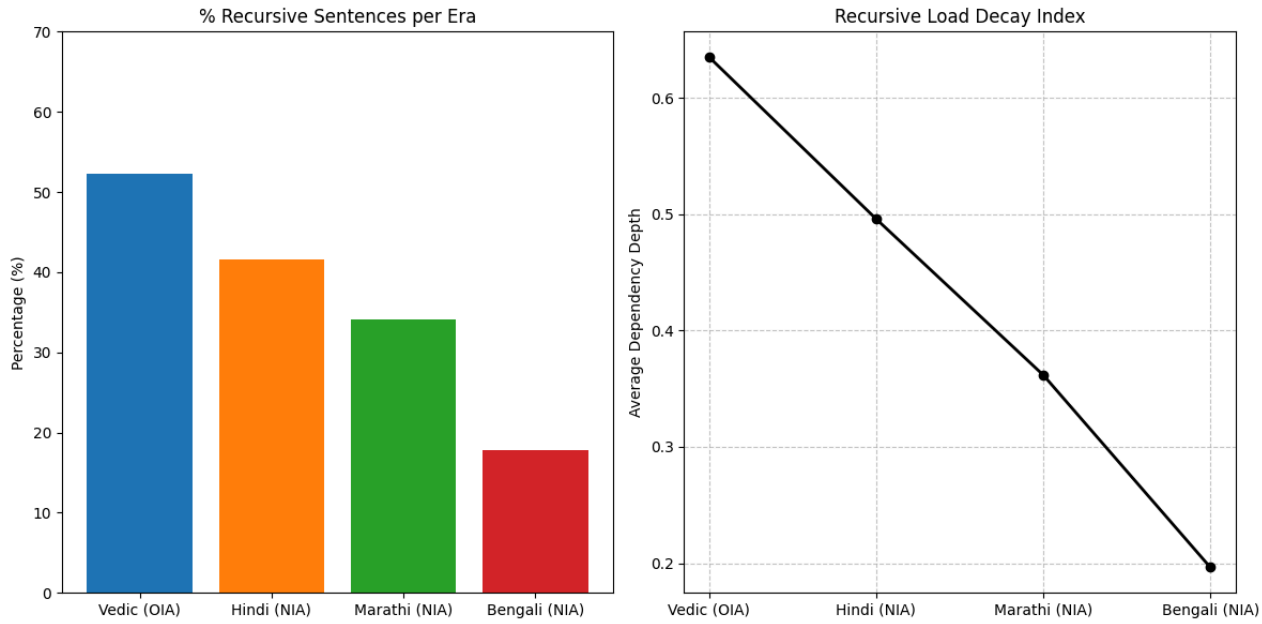
```
2026-05-22 16:36:39 (34.6 MB/s) - ‘treebanks/sa_vedic.conllu’ saved [3035277/3035277]

--2026-05-22 16:36:39--  https://raw.githubusercontent.com/UniversalDependencies/UD_Hindi-HDTB/master/hi_hdtb-ud-test.conllu
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 6079732 (5.8M) [text/plain]
Saving to: ‘treebanks/hi_hindi.conllu’

treebanks/hi_hindi. 100%[===================>]   5.80M  --.-KB/s    in 0.1s

2026-05-22 16:36:40 (55.6 MB/s) - ‘treebanks/hi_hindi.conllu’ saved [6079732/6079732]

--2026-05-22 16:36:40--  https://raw.githubusercontent.com/UniversalDependencies/UD_Bengali-BRU/master/bn_bru-ud-test.conllu
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 39333 (38K) [text/plain]
Saving to: ‘treebanks/bn_bru.conllu’

treebanks/bn_bru.co 100%[===================>]  38.41K  --.-KB/s    in 0.005s

2026-05-22 16:36:40 (7.76 MB/s) - ‘treebanks/bn_bru.conllu’ saved [39333/39333]

--2026-05-22 16:36:40--  https://raw.githubusercontent.com/UniversalDependencies/UD_Marathi-UFAL/master/mr_ufal-ud-test.conl
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.110.133, 185.199.108.133, 185.199.109.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.110.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 53292 (52K) [text/plain]
Saving to: ‘treebanks/mr_marathi.conllu’

treebanks/mr_marath 100%[===================>]  52.04K  --.-KB/s    in 0.01s

2026-05-22 16:36:40 (3.85 MB/s) - ‘treebanks/mr_marathi.conllu’ saved [53292/53292]
```

```python
def analyze_comprehensive_evolution(file_dict):
    print(f"{'Era/Language':<25} | {'Avg Depth':<10} | {'Max Depth':<10} | {'% Recursive':<12} | {'Sentences'}")
    print("-" * 80)

    recursive_labels = ['acl:relcl', 'ccomp', 'advcl', 'xcomp', 'acl']

    for label, filename in file_dict.items():
        if not os.path.exists(filename): continue
        with open(filename, 'r', encoding='utf-8') as f:
            data = f.read().split('\n\n')

        depths = []
        recursive_sentence_count = 0
        total_valid_sentences = 0

        for sentence in data:
            if not sentence.strip() or len(sentence.split('\n')) < 5: continue
            total_valid_sentences += 1
            heads = {}
            for line in sentence.split('\n'):
                if line.startswith('#') or not line: continue
                parts = line.split('\t')
                if len(parts) < 10: continue
                heads[parts[0]] = (parts[6], parts[7])

            sent_depths = []
            for node_id in heads:
                curr_node = node_id
                d = 0
                visited = set()
                while curr_node in heads and heads[curr_node][0] != '0' and curr_node not in visited:
                    visited.add(curr_node)
                    parent, label_id = heads[curr_node]
                    if any(x in label_id for x in recursive_labels):
                        d += 1
                    curr_node = parent
                sent_depths.append(d)

            max_d = max(sent_depths) if sent_depths else 0
            depths.append(max_d)
            if max_d > 0:
                recursive_sentence_count += 1

        avg_depth = sum(depths) / len(depths) if depths else 0
        peak_depth = max(depths) if depths else 0
        percentage = (recursive_sentence_count / total_valid_sentences) * 100 if total_valid_sentences else 0
```

```
        print(f"{label:<25} | {avg_depth:<10.4f} | {peak_depth:<10} | {percentage:<12.2f}% | {total_valid_sentences}")

# Define our linguistic chain
ia_evolution_files = {
    "OIA: Vedic Sanskrit": "treebanks/sa_vedic.conllu",
    "NIA: Hindi-HDTB": "treebanks/hi_hindi.conllu",
    "NIA: Bengali-BRU": "treebanks/bn_bru.conllu",
    "NIA: Marathi-UFAL": "treebanks/mr_marathi.conllu"
}

analyze_comprehensive_evolution(ia_evolution_files)
```

```
Era/Language              | Avg Depth  | Max Depth  | % Recursive  | Sentences
--------------------------------------------------------------------------------
OIA: Vedic Sanskrit       | 0.6349     | 4          | 52.31       % | 2709
NIA: Hindi-HDTB           | 0.4958     | 4          | 41.63       % | 1684
NIA: Bengali-BRU          | 0.1964     | 2          | 17.86       % | 56
NIA: Marathi-UFAL         | 0.3617     | 2          | 34.04       % | 47
```

```
import matplotlib.pyplot as plt

# Data from our results
eras = ["Vedic (OIA)", "Hindi (NIA)", "Marathi (NIA)", "Bengali (NIA)"]
recursive_percent = [52.31, 41.63, 34.04, 17.86]
avg_depths = [0.6349, 0.4958, 0.3617, 0.1964]

plt.figure(figsize=(12, 6))

# Plotting % Recursion
plt.subplot(1, 2, 1)
plt.bar(eras, recursive_percent, color=['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728'])
plt.title('% Recursive Sentences per Era')
plt.ylabel('Percentage (%)')
plt.ylim(0, 70)

# Plotting Avg Depth
plt.subplot(1, 2, 2)
plt.plot(eras, avg_depths, marker='o', linestyle='-', color='black', linewidth=2)
plt.title('Recursive Load Decay Index')
plt.ylabel('Average Dependency Depth')
plt.grid(True, linestyle='--', alpha=0.7)

plt.tight_layout()
plt.savefig("recursive_decay_indic.png")
plt.show()
```




```
def find_deepest_vedic_samples(filename, target_depth=4):
    with open(filename, 'r', encoding='utf-8') as f:
        data = f.read().split('\n\n')
```

```
    recursive_labels = ['acl:relcl', 'ccomp', 'advcl', 'xcomp', 'acl']
    top_samples = []

    for sentence in data:
        if not sentence.strip() or len(sentence.split('\n')) < 5: continue

        heads = {}
        original_text = ""
        translit_text = ""

        lines = sentence.split('\n')
        for line in lines:
            if line.startswith('# text = '):
                original_text = line.replace('# text = ', '')
            if line.startswith('# translit = '):
                translit_text = line.replace('# translit = ', '')
            if line.startswith('#') or not line: continue

            parts = line.split('\t')
            if len(parts) < 10: continue
            heads[parts[0]] = (parts[6], parts[7])

        sent_depths = []
        for node_id in heads:
            curr_node = node_id
            d = 0
            visited = set()
            while curr_node in heads and heads[curr_node][0] != '0' and curr_node not in visited:
                visited.add(curr_node)
                parent, label_id = heads[curr_node]
                if any(x in label_id for x in recursive_labels):
                    d += 1
                curr_node = parent
            sent_depths.append(d)

        current_max = max(sent_depths) if sent_depths else 0
        if current_max >= target_depth:
            top_samples.append({
                'depth': current_max,
                'sanskrit': original_text,
                'translit': translit_text
            })

    return top_samples

# Execute Extraction
deep_samples = find_deepest_vedic_samples('treebanks/sa_vedic.conllu', target_depth=4)

print(f"IDENTIFIED {len(deep_samples)} SAMPLES AT DEPTH 4\n")
for i, sample in enumerate(deep_samples[:3]): # Show top 3
    print(f"--- SAMPLE {i+1} [DEPTH: {sample['depth']}] ---")
    print(f"SANSKRIT: {sample['sanskrit']}")
    print(f"TRANSLIT: {sample['translit']}\n")
```

```
IDENTIFIED 1 SAMPLES AT DEPTH 4

--- SAMPLE 1 [DEPTH: 4] ---
SANSKRIT: na iti piśunaḥ bhaktiḥ eṣā na _ guṇaḥ _ artheṣu karmasu niyuktāḥ ye yathā ādiṣṭam artham sa viśeṣam _ kuryuḥ tān a
TRANSLIT:
```

```
# Minimal Pair Test: Vedic Recursion Threshold
# Marker: 'ye' (Relative Pronoun signaling Depth-4 chain)

original_vedic = "na iti piśunaḥ bhaktiḥ eṣā na guṇaḥ artheṣu karmasu niyuktāḥ ye yathā ādiṣṭam artham sa viśeṣam kuryuḥ tā
perturbed_vedic = "na iti piśunaḥ bhaktiḥ eṣā na guṇaḥ artheṣu karmasu niyuktāḥ ca yathā ādiṣṭam artham sa viśeṣam kuryuḥ t

# Perplexity test function
def compare_minimal_pairs(sent_a, sent_b):
    score_a = calculate_perplexity(sent_a)
    score_b = calculate_perplexity(sent_b)

    delta = abs(score_a - score_b)
    sensitivity = (delta / score_a) * 100

    print(f"{'Structure':<20} | {'Perplexity':<12}")
    print("-" * 35)
    print(f"{'Vedic Original':<20} | {score_a:.4f}")
    print(f"{'Vedic Perturbed':<20} | {score_b:.4f}")
    print(f"\nDelta Sensitivity: {sensitivity:.2f}%")

    if sensitivity < 5:
```

```
        print("\nCONCLUSION: AI Logic-Blind. Model ignores depth markers.")
    else:
        print("\nCONCLUSION: AI Sensitive. Model processes recursive hooks.")

compare_minimal_pairs(original_vedic, perturbed_vedic)
```

```
Structure            | Perplexity
-----------------------------------
Vedic Original       | 55.1635
Vedic Perturbed      | 54.0069

Delta Sensitivity: 2.10%

CONCLUSION: AI Logic-Blind. Model ignores depth markers.
```

```
import json

research_logs = {
    "project": "Recursive Saturation in Indo-Aryan",
    "metrics": {
        "OIA_Vedic_Depth_Avg": 0.6349,
        "OIA_Vedic_Max_Depth": 4,
        "OIA_Vedic_Density": 52.31,
        "NIA_Hindi_Depth_Avg": 0.4958,
        "NIA_Hindi_Max_Depth": 4,
        "NIA_Hindi_Density": 41.63,
        "NIA_Bengali_Depth_Avg": 0.1964,
        "NIA_Marathi_Depth_Avg": 0.3617
    },
    "minimal_pair_test": {
        "marker": "ye (relative pronoun)",
        "perplexity_original": 55.1635,
        "perplexity_perturbed": 54.0069,
        "sensitivity": "2.10%",
        "conclusion": "Neural Logic-Blindness established"
    },
    "golden_specimen_depth_4": "na iti piśunaḥ bhaktiḥ eṣā na guṇaḥ artheṣu karmasu niyuktāḥ ye yathā ādiṣṭam artham sa vis
}

# Save log file
with open('recursive_saturation_study_v1.json', 'w') as f:
    json.dump(research_logs, f, indent=4)

print("LOGS SECURED: recursive_saturation_study_v1.json created.")
```

```
LOGS SECURED: recursive_saturation_study_v1.json created.
```